

W1 DATA SCIENCE CAPABILITIES
Data science: domain/business knowledge + statistics/analytics + computing/IT; goal is useful decisions, not only accurate models.
CRISP-DM business understanding → data understanding → preparation → modelling → evaluation → deployment/monitoring. Evaluation asks whether the model meets business goals, bias/fairness, validity, risk.
Capabilities: identify data sources/types/quality, formulate probabilistic model, estimate parameters by optimisation, implement Python algorithms, communicate to technical/nontechnical users.
Problem types: regression continuous y ; classification discrete label; clustering/segmentation unlabeled; anomaly detection unusual observations; recommendation ranking; imputation filling missing values.
Business metric: choose objective around costs: disease/fraud → recall; spam/manual review → precision; marketing lift/ranking → PR-AUC/top-k; regulated decisions need interpretability/fairness.
Data types: continuous, discrete, binary, nominal, ordinal, text, time/date.
Prime-valued observations are discrete, ordinal has order but unequal numeric meaning.
4W: volume, velocity, variety, veracity/validity depending source, exam wording may treat “great value” as not one of IBM 4V if list is volume/velocity/variety/veracity.
Ethics: privacy, consent, representativeness, bias, explainability, feedback loops, monitoring after deployment. High validation score does not remove ethical risk. **Cynifem:** obvious-sense-category-respond; complicated-sense-analysis-respond; complex-probe-sense-probe; chaotic-act-to-business-respond. Rapid changing environment often chaotic.
Capability: ability to connect tool to sense process: define target, collect reliable data, choose baseline/model, evaluate with suitable metric, deploy with monitoring. **Baseline:** always compare to simple rule: majority class, mean prediction, simple segmentation, or business heuristic. Complex model must justify extra cost.
Knowledge discovery: classification, regression/prediction, clustering, pattern recognition, anomaly detection, association/recommendation. Imputation is usually data preparation, not a final discovery task.

Decision loop: single-loop asks “are we doing things right?”; double-loop asks “are we doing the right things?” and may change objective/assumption.
Marketing: goals: segmentation improves relevance/personalisation; A/B tests compare treatments; response/uplift model targets persuadable users; fatigue appears when repeated messages reduce engagement.
Success metric: align with business decision and timing. Referral rate can be business-aligned + real-time if tracked continuously; offline survey may be aligned but delayed.
Data product risk: business model can create feedback loop: decisions change future data, so monitoring must include input drift, prediction drift, and outcome drift.
Exam trap: “AI/ML” answer alone is inadequate; name data, target, method, metric, deployment action, and limitation in one compact paragraph.

W2 DATA LIFECYCLE
Lifecycle: acquire → store → clean → integrate → transform → model → evaluate → deploy. Record assumptions and transformations for reproducibility.
Quality issues: missing salary/values, duplicate records, inconsistent coding (M/Male), impossible ranges, unit mismatch, leakage, nonunique IDs, outliers, wrong joins.
Split rule: split before fitting/preprocessing when measuring performance. Fit imputer/scaler/vectorizer/PCA on train only; transform validation/test with train-fitted object.
Missingness: MCAR unrelated to data; MAR depends on observed variables; MNAR depends on unobserved/missing value. Deletion can bias if MCAR; impute inside CV folds.
Train/val/test: train fits parameters; validation tunes model/hyperparameters/threshold; test estimates final unseen performance. Never tune on test. Stratify classification splits if imbalanced.
Leakage traps: vectorise/scalar/PCA before split; use test labels for threshold; duplicate customers across train/test; impute using full-data mean; future information.
Data prep vs modelling: normalization, missing handling, encoding and visualisation are prep/understanding; hyperparameter tuning is modelling/validation, not data preparation.

Categorical encoding: one-hot nominal; ordinal only for real order; high-cardinality one-hot increases dimension/variance/computation; drop one level with intercept or regularise.
Join/ID traps: duplicated IDs can leak customers into both train/test; wrong joins duplicate rows; inconsistent primary keys make summaries wrong. Audit row counts after joins.
Imputation: mean/median for numeric, mode/ “missing” for categorical. Model-based imputation if justified. Add missingness flag if absence itself informative.
Pipeline rule: all learnt preprocessing parameters are trained on train folds only; use sklearn Pipeline to make CV leakage harder.
Sampling: simple random, stratified, cluster, convenience. Biased sample makes low test error misleading if deployment population differs.
Documentation: keep data dictionary, units, coding scheme, missing-value codes, data cutoffs, target definition and exclusion rules. Reproducibility is part of lifecycle.
Target leakage: predictors measured after the outcome, future aggregates, post-treatment variables, duplicate labels, or engineered target proxies. Remove or time-cut correctly.
Split types: random split for iid; stratified for rare classes; grouped split for repeated customers; time split for temporal prediction. Wrong split overstates generalisation.
Data governance: access control, anonymisation/pseudonymisation, retention, audit logs. Do not keep raw personal data in modelling artifact unless needed.
Cleaning rule: keep original raw file immutable; create cleaned feature table by scripted transformations so results can be reproduced and audited.

W3 EXPLORATORY ANALYSIS
EDA goals: understand distributions, outliers, missingness, relationships, class balance, skew, transformations, business plausibility. Plot before modelling.
Central tendency/spread: mean $\bar{x} = \frac{1}{n} \sum x_i$, median robust centre, mode most common; variance $s^2 = \frac{1}{n-1} \sum (x_i - \bar{x})^2$ measures spread, not centrality.
Cov/corr: $\text{Cov}(X, Y) = E[(X - \mu_X)(Y - \mu_Y)]$; sample $s_{xy} = \frac{1}{n-1} \sum (x_i - \bar{x})(y_i - \bar{y})$; $\rho = s_{xy}/(s_x s_y) \in [-1, 1]$. Correlation ≠ causation.
Skew/outliers: log transform positive skew; cap/winorise only with rationale; use robust loss/median/MAE; remove only data errors or justified anomalies.
Visual diagnostics: hist/boxplot for distribution; scatter for relationship; residual-vs-fitted for regression assumptions; bar chart for categorical counts.
Outlier/correlation: use PCA for multicollinearity clues.
Residual plot answer: not correctly specified if curvature, funnel spread, parallel bands, systematic pattern, outliers. Violates linearity/constant variance/independence/missing group effect.
Fix residuals: add nonlinear terms/interactions/group dummy, transform y or x , weighted/robust regression, collect missing variables, check outlier influence.
Distribution shift: model trained on one distribution can fail when deployment distribution changes; learned boundary may not match new population. Monitor drift and recalibrate/retrain.
Outlier vocabulary: outlier unusual y ; leverage unusual x ; influential point changes fitted model if removed. Do not delete without data/business reason.
EDA exam wording: “The plot suggests ..., therefore assumption ... may fail; I would

check ... and consider ...”. State both evidence and implication.
Skew transform: log positive skew, stabilises variance and makes multiplicative effects additive; back-transform predictions carefully.
Q-Q plot: points above line in upper tail & below in lower tail suggests heavy tails; right tail bending upward suggests positive skew. Use with histogram, not alone.
Correlation caveat: nonlinear relationship can have low Pearson correlation; outliers can dominate correlation. Spearman rank correlation helps monotone nonlinearity.
Class balance: inspect positive rate and subgroup distribution. Accuracy is weak when majority class dominates; baseline majority accuracy must be reported.
Multicollinearity clue: highly correlated predictors, unstable coefficient signs, large SEs. Ridge/PCA/remove duplicate features can help prediction.
Residual assumptions: residual mean near 0, no systematic pattern, constant spread, independence, approximate normality for inference. Prediction can still work when inference assumptions fail.
Influence check: compare fit with/without suspicious point; leverage comes from unusual x , residual from unusual y , influence combines both.

W4 MATH + SCIENTIFIC COMPUTING
Shapes: $X \in \mathbb{R}^{n \times p}$, $y \in \mathbb{R}^n$, $B \in \mathbb{R}^p$, $\hat{y} = X\hat{\beta}$, $\hat{y} = y - \hat{y}$. Add intercept: $X_0 = [1, X]$, $\hat{\beta} = (X^T X)^{-1} X^T y$ if nonsingular; hat matrix $H = X(X^T X)^{-1} X^T$, so $\hat{y} = Hy$.
OLS normal eqs: minimise $\|y - X\hat{\beta}\|_2^2$; gradient $-2X^T(y - X\hat{\beta}) = 0 \Rightarrow X^T \hat{\epsilon} = 0$. With intercept, $\sum_i \hat{\epsilon}_i = 0$ and $\sum_i y_i \hat{\epsilon}_i = 0$.
Orthogonality: $\hat{\beta}_1 \perp \hat{\beta}_2 \iff X^T \hat{\epsilon} = 0$. Proof questions usually need intercept for residual sum-zero.
Invertibility: $X^T X$ singular if exact collinearity, $p > n$, duplicate features/intercept, one regressor multiple of another, $\log(x) = 2 \log(x)$ for $x > 0$.
Distance/inner: $\|x - \hat{x}\|_2^2 = \sum (x_i - \hat{x}_i)^2$; $a^T b = \sum a_i b_i$; nested scalar $\langle (a, b), c \rangle = (a^T b) \cdot c$.
Derivatives: $\nabla(b^T \theta) = b$; $\nabla(A^T \theta) = (A + A^T)/2$ if symmetric $= 2A\theta$. MSE gradients: $\partial_{\beta_0} = -2n^{-1} \sum \epsilon_i$, $\partial_{\beta_1} = -2n^{-1} \sum x_i \epsilon_i$.
Numpy: dot matrix multiply; $*$ elementwise with broadcasting; $A.T$ transpose; np.c_[cols] X intercept. Prefer `solve/linalg` over explicit inverse.
Broadcasting: compare trailing dims; equal or 1 works. $(1, 7)$ with $(3, 7)$ OK; $(5, 1)$ with $(3, 5)$ OK; $(4, 3)$ with $(1, 3)$ OK; $(3, 1)$ with $(1, 4)$ with $(3, 5)$ fails.
Axis: `axis=0` reduces rows → column stats; `axis=1` reduces columns → row stats. X shape: (0) rows, X shape: (1) columns.
Code gradient: vectorised MSE: `yhat=X@beta; e=y-yhat; grad=-2*n*X.T@e`; update `beta+=grad*grad`.
Covariance matrix: symmetric; diagonal entries are variances; entries can be negative; correlation matrix has diagonal 1.
Projection: H is symmetric/idempotent: $H^T = H$, $H^2 = H$ when $X^T X$ invertible.
Residual maker $M = I - H$; $\hat{\epsilon} = My$.
Rank: full column rank means no exact linear dependence among columns. If rank deficient, inverse fails; use least squares pseudoinverse or regularise.
Norm facts: squared distance between $x = (5, 1, 4)$ and $z = (2, 3, 1)$ is $3^2 + (-2)^2 + 3^2 = 22$. Centered variance can be $n^{-1} \|x\|^2$ depending convention.
Plot caveat: linearity. Pandas/Numpy comparisons need parentheses: `(df.A > 2) & (df.B < 5)`; Python `and` fails on non-boolean.
Clip: `np.clip(a, lo, hi)` caps below/above. Example `clip([-1, 1.2, 3.7, 5], 2, 4)` gives `[2, 2, 2, 4]`.
Shape drill: row vector vs column vector matters: `x.reshape(-1, 1)` creates one feature column; forgetting it can make sklearn treat shape incorrectly.

W5 CLUSTER ANALYSIS
k-means: choose k , initialise centroids, assign each point to nearest centroid, update centroids to mean of assigned points, repeat until stable. Objective $\sum_i \min_{j \in \{1, \dots, k\}} \|x_i - \mu_j\|^2$.
Complexity: one iteration $O(nk)$ with fixed $d, O(nk)$. Multiple starts/iterations multiply cost.
Disadvantages: sensitive to scaling, outliers, initialisation, k , spherical/equal-size cluster assumption. Run many n_{start} ; inspect cluster sizes of centroids.
Code: k elbow/inertia stability; silhouette better; standard bias; standard across random starts; ARI/external labels if available; domain usefulness.
Unsupervised validation: hard because no target labels. Holding out data alone does not provide accuracy; use internal metrics, stability, interpretability, business actionability.
PCA: linear projection to orthogonal components maximising variance; reduces dimension/noise/collinearity, but standard PCA does not capture nonlinear relationships.
PCA calculation: centre $X_0 = X - X_0$; covariance $S = (n-1)^{-1} X_0^T X_0$ shape (p, p) ; eigendecompose: sort eigenvalues descending; transform $Z = X_0 V_k$.
PCA code: `X_mean=X.shape(0); X=X.shape(1, p); np.argsort(eigenvalues)[::-1]` returns descending indices. Fit PCA/scaler on train only.
Silhouette: $c = (b - a) / \max(a, b)$; near 1 good separation, near 0 overlap, negative likely wrong cluster. Use with inertia and domain sense.
Cluster interpretation: report centroid profile, size, separation, and business action. Clusters are not ground-truth labels.
k-means code: after assignment to nearest centroid, next step is update centroid to mean of assigned points; empty clusters may be reintitiated.
k-means distance: assign x to centroid with smallest squared Euclidean distance; square root unnecessary for comparison. Scaling changes distance weighting.
Noise impact: outliers can pull centroids or be absorbed into nearest clusters; with larger k , noise may form small artificial clusters.
Elbow: inertia always decreases as k rises; choose point where marginal gain flattens, plus interpretability and business sense.
PCA variance: explained variance ratio $\lambda_1 / \sum \lambda_i$; choose components by cumulative variance or validation performance. Components may have low interpretability.
Standardisation before PCA: use when variables have different units; otherwise high-variance unit dominates first component. Fit scaler on train only.
PCA false statement: standard PCA is linear; it does not directly capture nonlinear manifolds unless kernel/nonlinear variant used.
Segmentation business: capture clustering, name segments by centroid pattern, size and actionability; do not claim causal differences.

W6 REGRESSION
Linear model: $Y = \beta_0 + \beta_1 x + \epsilon$, $E(\epsilon|x) = 0$, $\text{Var}(\epsilon|x) = \sigma^2$. Conditional MSE minimiser is $E(Y|x) = \beta_0 + \beta_1 x$.
For any Y , argmin $_{\beta}$ $E[(Y - \hat{y})^2] = E[Y]$ because $E[(Y - \hat{y})^2] = \text{Var}(Y - \hat{y}) + [E(Y - \hat{y})]^2$.
Prediction/inference: point $\hat{y} = x^T \hat{\beta}$; mean CI narrower than prediction interval because PE includes new noise. Use 95% CI approx $\pm 1.96 E(\hat{\beta}_j)$.
Weighted LS: minimise $\sum w_i (y_i - \beta_0 - \beta_1 x_i)^2$; normal eqs $\sum w_i x_i = 0$, $\sum_i w_i x_i \epsilon_i = 0$; matrix $\hat{\beta} = (X^T W X)^{-1} X^T W y$. Equal weights cancel → OLS.
Design matrix: for $y = \beta_0 + \beta_1 \sqrt{x} + \beta_2 x^2 + \beta_3 x^3 + \beta_4 x^4 + \beta_5 (x^2/x)$ row is $(\epsilon_1, \sqrt{x}, x, x^2, x^3, x^2/x)$; ϵ_i evaluate indicators rowwise.
Polynomial features: capture curvature; high degree can overfit and explode dimension. Choose degree by validation/CV, not training error.
Regularisation: Ridge $RSS + \lambda \sum \beta_j^2$ shrinks, handles multicollinearity, no exact zeros; Lasso $RSS + \lambda \sum |\beta_j|$ sparse; Elastic Net mix. Scale before penalties.
Residual assumptions: mean zero, independent, no autocorrelation, constant variance, approx normal for inference. Autocorrelated residuals are a violation, not

usual assumption.
Ridge form: $\hat{\beta}_\lambda = (X^T X + \lambda I)^{-1} X^T y$; do not penalise intercept in standard use. Larger λ raises bias, lowers variance.
Q-Q plot: points above line in upper tail & below in lower tail suggests heavy tails; right tail bending upward suggests positive skew. Use with histogram, not alone.
WLS meaning: weights often inverse variance; large weight = observation trusted more. Equal weights reduce to OLS.
Unbiased slope: if $y_{ij} = \beta_0 + \beta_1 x_{ij} + \epsilon_{ij}$, $E(\epsilon_{ij}|X) = 0$, then $E(\hat{\beta}_1|X) = \beta_1$. Bias arises from omitted variables/correlation with error.
MLE-OLS: with iid Gaussian errors and constant variance, maximizing likelihood is equivalent to minimizing RSS. Without normality, OLS may still be unbiased under exogeneity.
Log model: if log $y = x^T \beta + \epsilon$, median prediction is $\exp(x^T \hat{\beta})$; mean needs normal correction if $\text{Var}(\epsilon)$ known.
Nonlinear basis: linear in parameters can include $\sqrt{x}$, x^2 , x^3 , $g(x)$. Design row uses transformed features, not raw x only.
Overfitting symptom: low training error + high validation/test error. Remedies: simpler model, ridge/lasso, more data, feature pruning, CV, early stopping.
Regression metrics: MSE/RMSE penalise large errors; MAE robust; $R^2 = 1 - RSS/TSS$ compares to mean baseline and can be negative on test.
Residual proof: with intercept, first normal equation gives $1^T \hat{\epsilon} = 0$; since $y = X\hat{\beta}$, $\hat{\epsilon} = \hat{\beta}^T X^T \hat{\epsilon} = 0$.

W7 FEATURE ENGINEERING
Feature engineering: create useful predictors from raw variables: scaling, transforms, interactions, polynomials, bins, text vectors, dates, domain ratios/indicators. Adds signal but can add noise/leakage.
Scaling: standardise $(x - \mu_{\text{train}})/s_{\text{train}}$; min-max $(x - \min)/(max - \min)$. Crucial for gradient descent, PCA, KNN, SVM, regularised/logistic models; trees less sensitive.
One-hot: good for nominal categories; high-cardinality variables can hugely increase dimension and overfit/computation. Consider grouping rare levels; target encoding with CV, hashing, or regularisation.
Transform: log for positive skew and multiplicative effects; polynomial/interactions for nonlinear relationships; indicator $(x > c)$ for threshold behaviour; splines if smooth curvature.
Feature selection: filters (correlation/chi-square), wrappers (RFE), embedded (lasso/step selection). PCA is feature extraction/dimensionality reduction, not supervised feature selection.
Polynomial degree code: split train/validation; loop candidate degrees; build PolynomialFeatures; optionally scale; fit appropriate classifier/regressor; score validation; return max accuracy/min loss degree.
Overfitting fix: high train performance + poor validation = overfitting. Fix simpler model, regularisation, feature selection, CV, more data, early stopping, leakage audit.
Exam wording: adding features can hurt because new features may add dimensionality/noise without signal; more expressive power does not guarantee generalisation.
Summary trap: with intercept, one-hot all levels causes collinearity. Drop one reference level or rely on regularisation for implementation handling.
Date/text features: extract month/day/year/counts; vectorise text with counts/TF-IDF; fit feature extractor on train only.
Feature selection caveat: selecting features using full data leaks test information; selection must occur inside CV pipeline.
Interactions: include $x_1 x_2$ when effect of one variable changes with another. Interpret main effects conditionally, not as universal effects.
Binning: converts continuous variable to ordinal indicators; can model thresholds but loses information and creates arbitrary cut points.
Categorical rare levels: combine rare categories into “other” before one-hot to reduce sparsity; learn rule on train only.
Target encoding: replace category by mean target within train folds; powerful for high-cardinality but severe leakage if computed on full data.
Tree note: tree models usually do not need scaling, but log transforms can still help skew/outliers and interpretation. Distance/gradient/PCA models do need scaling.
Feature store idea: centralises reusable features and definitions; prevents inconsistent train/test leakage.
Assignment answer: explain why feature was created and how it changes model capacity/risk; never say “more features always better”.

W8 CLASSIFICATION
Logistic: $p = P(Y = 1|x) = \sigma(\eta)$, $\eta = \beta_0 + \beta_1 x$, $\sigma(z) = 1/(1 + e^{-z})$. Odds $p/(1 - p) = e^\eta$, log-odds = η . Plug-in: if $\eta = -1 + 2(0.3) - 0.1 = -0.5$, $p = \sigma(-0.5) \approx 0.378$. Threshold 0.5 boundary $\eta = 0$ only if equal costs.
Interpret: adding all other features equivalent to intercept. Centering/scaling changes intercept but does not generally remove need. Logistic parameters estimated by likelihood, not OLS.
Softmax: $P(Y = k|x) = \exp(x^T \beta_k) / \sum_j \exp(x^T \beta_j)$; alternative one-vs-rest.

$\hat{y} = 1$	$\hat{y} = 0$	
$y = 1$	TP	FP
$y = 0$	FN	TN

Metrics: Accuracy $(TP + TN)/N$; Precision $TP/(TP + FP)$; Recall $TP/(TP + FN)$; Specificity $TN/(TN + FP)$; FPR $FP/(FP + TN)$; F1 $= 2TP/(2TP + FP + FN)$.
Metric choice: precision if FPR costly; recall if FN costly; F1 balances; specificity excludes negatives; PR curve more informative for rare positives; ROC plots TPR vs FPR; AUC = area under positive ranks above random negative.
Imbalance: class weights, oversampling, undersampling, SMOTE, threshold tuning, stratified CV, report PR-AUC/F1/CI/specificity, not accuracy alone.
Confusion drill: $\text{f1} = \text{fpr}$; $\text{tn} = \text{tn}$; $\text{tp} = \text{tp}$; $\text{fp} = \text{fp}$; $\text{fn} = \text{fn}$ — specificity, $\text{f1} = 2TP/(P(R) + 2TP/(P(R) + FP + FN))$.
Threshold: predict $\text{proba} = 1$, $\{1\}$ gives scores; compare to validation-chosen threshold. Default 0.5 may be bad under asymmetric costs.
Logistic coefficient: e^{β_1} multiplies odds for one-unit x_1 change holding others fixed; probability change is not constant.
FPR drill: $\text{FPR} = FP/(FP + TN)$. If actual negatives are 6 and two are false positives, $\text{FPR} = 2/6 = 0.333$.
All-zero predictions: if positives exist, recall = 0; precision undefined because $TP + FP = 0$, often reported after by convention.
ROC vs PR: ROC can leak optimistic under rare positives; PR focuses positive class and is better for fraud/disease/rare response.
Classification one-vs-rest fits binary classifiers; softmax estimates class probabilities jointly. Macro-average treats classes equally; micro-average pools counts.
Decision tree split: classification uses impurity such as Gini/entropy; regression uses variance/SSE. Entropy high means mixed/impure node.
Logistic likelihood: maximise $\sum y_i \log p_i + (1 - y_i) \log(1 - p_i)$; parameters by numerical optimisation, not closed-form OLS.
Calibration vs ranking: AUC measures ranking; calibrated probability requires predicted risk to match empirical frequency. Thresholding needs business cost.

W9 MODEL EVALUATION
EPE: $EPE(x_0) = E_{y_0|x_0} E_{y'}[(y_0 - y')^2 | y_0]$. Expectation over training set/model

randomness and new response/noise, conditional on fixed x_0 .
Bias-variance-noise: $E[(Y - \hat{y})^2] = \sigma^2 + \text{Bias}(\hat{y})^2 + \text{Var}(\hat{y})$ at fixed x .
Capacity: low complexity → high bias/underfit; high complexity → high variance/overfit. Choose model minimising validation/test error, not training error.
K-fold CV: split into k folds; train on $k - 1$, validate on 1; average. LOOCV $k = n$ trains on $n - 1$ each time but costly and not always low variance. Stratified CV for class imbalance.
Bootstrap: sample n with replacement; $P(\text{not selected}) = (1 - 1/n)^n \rightarrow e^{-1} \approx 0.368$ OOB. Bagging uses bootstrap models to reduce variance.
Model closest question: plug x^* into deterministic part $E[y|x^*]$; compare $|y - y^*|$; ignore zero-mean noise if asked numerically.
Business evaluation: align metric with objective and cost; check subgroup errors/fairness; calibration for probabilities; stability under distribution shift; interpretability for deployment.
Confusion example: $y = [1, 1, 0, 0, 1, 1]$, $\hat{y} = [0, 0, 1, 0, 0, 1]$; TP=1, FP=1, FN=3, precision=1/2, recall=1/4, F1=1/3.
Calibration: predicted 0.8 should mean about 80% positives among similar cases. Ranking/accuracy can be good while probabilities are miscalibrated.
Test use: final test use once after model choice. If you repeatedly inspect test results, it becomes validation data.
Bootstrap OOB: not selected prob $(1 - 1/n)^n \rightarrow e^{-1}$; OOB examples can estimate bagging/random-forest error.
LOOCV: trains on $n - 1$ each time, usually low bias but high computation and not necessarily lower variance than k-fold. Poor for huge data.
Nested CV: outer loop estimates performance; inner loop tunes hyperparameters. Need when reporting honest model performance.
Generalisation gap: validation error - training error. Large positive gap suggests variance/overfit; both high suggests bias/underfit.
Ensembles: bagging/Random Forest reduces variance by averaging bootstrap models; boosting sequentially focuses errors and can reduce bias but overfit if too long.
Distribution shift: covariate shift $P(X)$ changes, label shift $P(Y)$ changes, concept drift $P(Y|X)$ changes. Recalibration/retraining depends on type.
Model selection: choose by validation metric aligned with objective, then inspect β coefficients. Beware of overfitting; simplest adequate model is easier to justify.
Uncertainty: validation score is an estimate; report confidence/variability via folds/bootstraps when comparing close models.

W10 OPTIMISATION
MLE: $L(\theta) = \prod_i p(y_i, \theta) \Rightarrow \ell = \sum_i \log p(y_i, \theta)$; maximise ℓ or minimise negative log-likelihood. State domain constraints.
Expt rate: $y_i \sim \text{Exp}(\lambda)$, $p(y|x) = \lambda e^{-\lambda y}$, $E[Y] = 1/\lambda$, $\ell = n \log \lambda - \lambda \sum y_i$; score $\partial \ell / \partial \lambda = \sum y_i$.
 $\lambda = 1/\mu$.
Negative log GD: $\ell = -n \log \theta + \theta \sum y_i$, gradient $-n/\theta + \sum y_i$. Projected update $\theta_{t+1} = \Pi_{(0, \infty)}[\theta_t - \eta_t(-n/\theta_t + \sum y_i)]$.
Normal-one slope: $y = \beta_0 + \epsilon$, $\epsilon_i \sim N(0, 1)$; $\ell = -\frac{1}{2} \log \text{var} - \frac{1}{2} \sum (y_i - \beta_0)^2$; score $\partial \ell / \partial \beta_0 = \sum (y_i - \beta_0)$; $\hat{\beta} = \sum y_i / \sum 1$.
Gradient descent: initialise θ_0 , compute $\nabla L(\theta_0)$, update $\theta_{t+1} = \theta_t - \eta_t \nabla L(\theta_t)$, stop by small loss/gradient or max iter. Large η diverges; small η slow. SGD: use one example/mini-batch gradient; noisy but cheap. For stochastic approximation, common sufficient step size $\sum \eta_t = \infty$, $\sum \eta_t^2 < \infty$.
Convexity: $f'' > 0$ strictly convex; e^{-x} strictly convex on $\mathbb{R}$; $|x|$ convex not strict; x^3 not convex on all $\mathbb{R}$; $-\log x$ convex only $x > 0$.
Argmin/min: for $f(x) = 3 + (x - 2)^2$, argmin $f = 2$, min $f = 3$. Argmin is location; min is value.
Jensen: for convex ϕ , $E[\phi(Y)] \geq \phi(EY)$; equality if degenerate/linear region. Useful for expectation/variance/prediction proofs.
Ascent/descent: maximise ℓ with $\theta + \eta \nabla \ell$; minimise NLL with $\theta - \eta \nabla \ell$. Keep positive parameters via projection/log-param.
MSE drill: targets $(2, 4, 6)$, preds $(1.5, 1.5, 1.5)$ gives residuals $(1, 5, 3, 4, 5)$ and MSE 10.
Power density rule: if $p(y) = \theta y^{a-1}$, $0 < y < 1$, $\ell = n \log \theta + (b - a) \sum \log y_i$; score $\partial \ell / \partial \theta = n \log y_i$; $\hat{\theta} = -n / \sum \log y_i$.
Hessian: second derivative negative at MLE for maximum; positive at minimiser. For exponential rate, $\ell'' = -n/\lambda^2 < 0$.
Step size: constant too large may oscillate/diverge; decreasing stochastic steps often reach $\sum \eta_t = \infty$, $\sum \eta_t^2 < \infty$.
Projection: $\Pi_{(0, \infty)}(x) = \max(x, 0)$, η in code, or parameterise $\theta = e^\alpha$ to maintain positivity.

Stopping: max iterations, small gradient norm, small parameter change, small objective change. Exam may count max-iteration and tolerance as two criteria.
Convex/concave: maximum concave log-likelihood or minimise convex NLL when possible; local minimum risk appears in nonconvex NN/objectives.
GD code: store old loss, compute gradient using current params, updates params, recomputes loss, breaks if abs `losses-old` < `tol`.
W11 BIG DATA SOLUTIONS
Pipelines: ingest → clean → feature store/vectorise → train → evaluate → deploy → monitor drift and feedback loops. Reproducibility matters: random seeds, versioned data/code, documented preprocessing.
Text mining: bag-of-words counts/frequencies ignore order/semantics; suitable for document classification where word frequency matters, weak for generation/summarisation.
TF-IDF: t_{ij} is # of times t_i appears in d_j ; $\text{idf}(t_i) = \log N/d(t_i)$; $t\text{-}f \cdot \text{idf}(t_i) = t_{ij} \cdot \text{idf}(t_i)$. If term appears in all docs, $\text{idf} = 1$, $t\text{-}f = 0.02$, $\text{idf} = 0.02 \log N \rightarrow 0$.
TF-IDF risk: rare type/spam token can dominate ranking; no synonyms/antonyms/order; depends on preprocessing.
Text pipeline: load → split → fit vectorizer on train → transform train/test → fit LogisticRegression/Naive Bayes/SVM → evaluate. Label length must equal matrix rows.
Recommendation: low accuracy/improvement: feature engineering, CV/grid search, better objective/metrics, representative data, ranking metrics, deep learning is not automatic fix.
Complexity order: $O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(n^3) < O(n^4) < O(2^n) < O(n!)$. For fixed k , $\alpha > 1$, $n(\log n)^k = o(n^\alpha)$.
Loop complexity: consecutive loops add; nested multiply. Sum n rows for p cols $O(np)$; p^2 dot products length n $O(n^2 p^2)$; SGD step with p features $O(p)$.
Time traps: TF-IDF ignores order, context, synonyms/antonyms; rare type/spam can dominate. Control preprocessing and inspect top-weighted tokens.
Big data pipeline: batch vs streaming; storage/compute choices should match velocity/volume; monitor data pipeline failures and drift.
Complexity trap: vectorised Numpy may be faster but asymptotic work remains; nested loops multiply even if written compactly.
Bag-of-words: vector counts/tokens/frequencies; ignores token order. Statement “counts number of words in a document” is less precise than counts occurrences per vocabulary term.
Vectorizer rule: fit `TfidfVectorizer` on training corpus, then `transform(new_corpus)`. Refitting on test changes vocabulary/columns and leaks.

Recommendation/uplift: treatment-control difference estimates lift. Persuadable if $P(Y = 1|T) > P(Y = 1|C)$; sleeping dog if treatment lowers response.
Scalability: memory, latency, fault tolerance, parallelism and data freshness matter in production, not just model accuracy.
Loop examples: loop over p columns $O(np)$; loop over p pairs $O(p^2)$; loop over p rows $O(pn)$; loop over p and n with dot length is $O(pn^2)$.
USYD 各科往届题库

Sparse text: high dimensional sparse matrix; conversion can exhaust memory.
Deployment: save preprocessing steps + production feature computation in diff.
W12 CASE STUDY
Sklearn split: `train_test_split(X, y, random_state=0)`; use Pipeline so preprocessing/steps are saved.
Normal eq code: `X=np.array([[1, 1, 0, 0], [1, 0, 1, 0], [0, 1, 1, 0], [0, 0, 1, 1]]); y=np.array([1, 0, 1, 1]); X_inv=np.linalg.pinv(X); yhat=X_inv.dot(X.T.dot(y))`.
Vectorised gradients: `yhat=X@beta; e=y-yhat; grad=-2*n*X.T@e`.
Shape: `axis=0` reduces rows → column stats; `axis=1` reduces columns → row stats. Shape mistakes common in PCA/covariance/code questions.
Confusion sklearn: `confusion_matrix(y_true, y_pred)` usually rows actual, columns predicted: `[TN, FP, FN, TP]` for labels=1; verify label order.
Pandas: `(df['C'] > 1) & (df['A'] > 1) & (df['B'] > 1)` elementwise. A=[1,2,3,4], B=[5,6,7,8] gives C=[6,8,10,12], sum=36.
Numerical safety: avoid explicit inverse for production; use `solve/linalg` if singular; remove collinear features or use ridge.
Report style: state objective, data prep, model, validation, metric, result, limitation, business recommendation. Include ethics/fairness where relevant.
Generalisation gap: validation error - training error. Model inside Pipeline; GridSearchCV tunes parameters like `max_depth`, `C_poly_degree`.
Class weight: `class_weight='balanced'` weights classes inverse to frequency; useful baseline for imbalanced classification.
Shape fixes: single feature often needs `reshape(-1, 1)`; `ravel()` flattens target; label length must match rows.
Polynomial code: `PolyFeatures(2)` gives `Pipeline(poly, clf)` with `PolynomialFeatures(2)` + `logistic`; score validation; keep best d .
PCA code: `X=X-X.mean(axis=0)`; `S=np.cov(Xc, rowvar=False)` or `Xc.T@Xc/(n-1)`; sort eigenvalues descending before projection.
OLS code: for `np.linalg.inv(X.T@X)` errors, likely singular matrix; prefer `np.linalg.lstsq(X, y, rcond=None)` or ridge.
Plot code: predicted vs actual scatter with 45-degree line; residual plot `p1=plt.scatter(yhat, y-yhat)` checks pattern/funnel/outliers.
Complexity answer: ignore constants/lower-order terms; for fixed k , α , compare asymptotic classes not numeric value at small n .

Exam writing: for proof, define objective, expand/simplify, take derivative/normal equation, state condition. For code, state shape and what each line computes. Final checklist: split before preprocessing; choose metric; compare baseline; tune on validation/CV; final test once; report limitations, leakage risk and business implication. Assignment design: formulate business questions as target y_i , unit of analysis, prediction time, allowable predictors, action, and metric. Wrong target timing creates leakage.
Data audit: before modelling, check row count, duplicates, missing map, impossible values, feature units, target prevalence, train/test overlap, and whether labels are measured after predictors.
Model card: intended use, training data, performance by subgroup, limitations, ethical risks, monitoring plan. Useful for nontechnical communication and governance.
Business recommendation: convert metric to action; who receives intervention, expected cost/benefit, threshold rationale, human review trigger, rollback rule.
Code robustness: set `random_state`, use Pipeline; keep preprocessing in fitted object; shape shapes; avoid chained assignment; inspect `X.shape()`, `sum()`.
Threshold report: default 0.5 is rarely final. Focus threshold on validation by cost matrix, recall constraint, capacity constraint, or expected profit. Do not tune threshold on test.
Ethics in project: include consent/privacy, bias from historical labels, subgroup error, explainability, intervention harm, and whether data subjects can contest decisions.
Common final traps: train-test leakage, comparing k -means inertia across scaled/unscaled data, interpreting odds as probability, using accuracy for rare positives, fitting vectorizer on test.
Proof mini-template: state dimensions; write objective; expand matrix form; take gradient; set to zero; verify convexity/second derivative; mention singular/constraint CAS.
Plot interpretation: name the pattern, link to assumption, state consequence, propose diagnostic/fix. Example: funnel residuals → heteroskedasticity → WLS/transform/robust SE.

One-line exam answer: “Because preprocessing was fitted before the split, validation error is optimistic; fit scaler/vectorizer inside CV Pipeline and evaluate once on untouched test.”